Exceptions, Assertions, and Logging



Do or do not.
There is no try.



But Master Yoda, we must try and catch exceptions in Java.









Like you, I do not.

THE IDEA

An exception is an object that reports a problem

1. Something goes wrong, so a method creates an exception object and throws it.
2. That method stops at once. The exception travels back up the chain of method calls, called the call stack.
3. The first catch block that matches handles it. The program then recovers and continues, or ends cleanly.
4. If nothing catches it, the program stops and prints a stack trace.

Why not return -1 or null? A special value can be ignored, or mistaken for a real answer. An exception cannot.

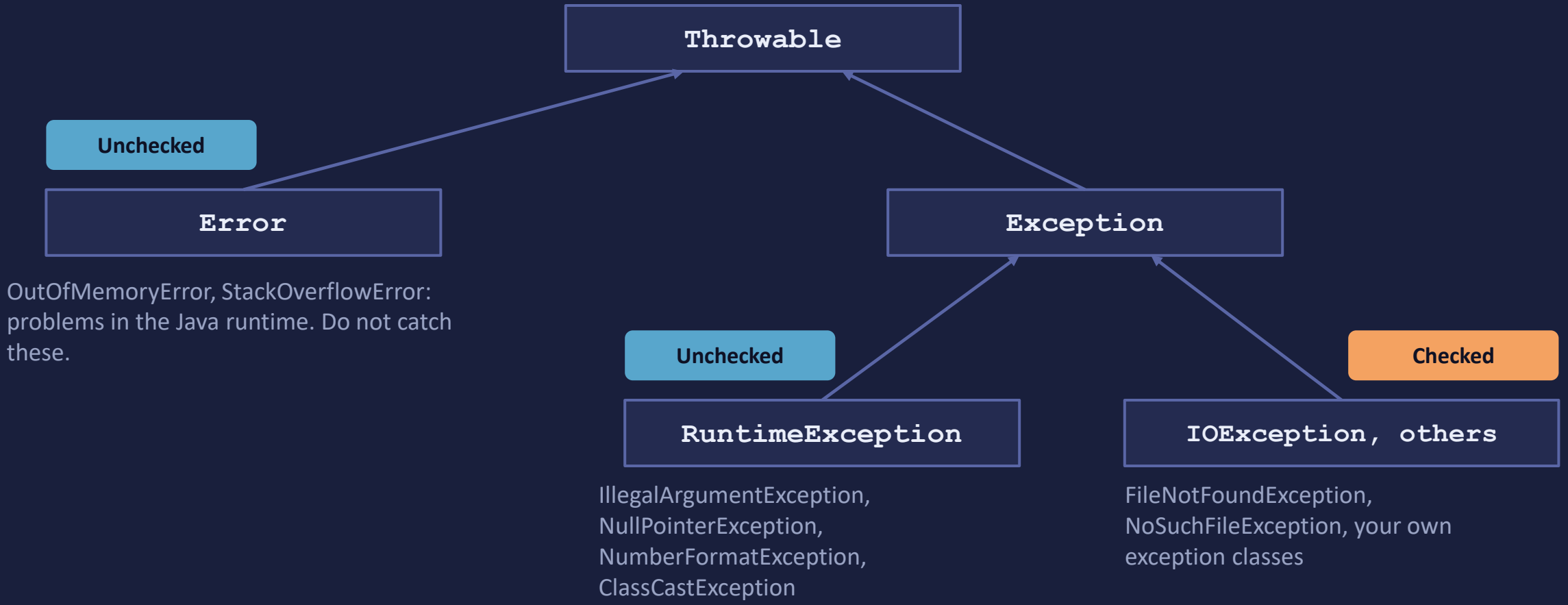


Not caught: the program stops



Caught: continue or end gracefully

Exceptions are classes in an inheritance hierarchy



Because these are classes, a catch block is polymorphic: catching IOException also catches every subclass of it.

The compiler tracks only checked exceptions

Checked

- The rule: catch it or declare it with throws. Otherwise the code does not compile.
- Usually the world is wrong: no amount of careful code guarantees a file exists.
- IOException, FileNotFoundException, ClassNotFoundException, SQLException

Unchecked

- No compiler rule. Methods may still document them with @throws.
- Usually the code is wrong: a bad index, an unexpected null, a bad cast. Fix the code.
- IllegalArgumentException, NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException

Common misconception: checked exceptions do not happen at compile time. Every exception happens while the program runs. The compiler only checks that you planned for the checked ones.

Six keywords and forms for handling exceptions

`throw`

Sends an exception object up the call stack. The current method stops immediately.

`throws`

In a method header, declares the checked exceptions the method passes to its caller.

`try and catch`

Runs code that might throw, then handles matching types. List the most specific type first.

`multi-catch`

One catch block for two unrelated types that need the same handling.

`finally`

Runs after try and catch every time, for cleanup. Never return from it.

`try-with-resources`

Closes files and other resources automatically, in reverse order, even after an exception.

Red cards send an exception on its way, amber cards handle it, and teal cards clean up.

Good exception design

1. Use a standard exception when one fits, such as IllegalArgumentException or IllegalStateException.
2. Write your own exception class when the caller needs a distinct type or extra information.
3. Choose checked for problems the caller can expect and recover from; unchecked for programming mistakes.
4. When you translate a low-level exception into your own, pass the original along as its cause.
5. Throw early, catch late: catch where the code knows what to do next, often inside a loop.
6. Never leave a catch block empty. A swallowed exception hides the problem.
7. Keep exceptions for exceptional circumstances. Never use them to make decisions or end loops.

Inheritance rule

An overriding method may throw fewer or narrower checked exceptions, never broader ones. Code that holds a superclass reference must never be surprised.

Constructors

A constructor that throws means the object is never created. That is how validation keeps bad objects from existing.

Check your assumptions; record what happened

Assertions

- State something that must be true if your code is correct.
- Off by default; turn them on with -ea while testing.
- Never for checking public arguments, and never with side effects.

Logging

- Replaces debugging print statements with leveled messages.
- Levels from SEVERE to FINEST; INFO and above show by default.
- Handlers send messages to the console, a file, or both.

Logging safely

Log4Shell (2021) was a flaw in the Apache Log4j 2 library, not in Java's built-in logging. Its lesson applies to every logger: logged text can come from an attacker. Clean line breaks out of untrusted input, never log passwords or other secrets, and keep logging libraries updated.